



PRESIDENCY UNIVERSITY

CSE322: Microprocessors Based System Design Laboratory

Lecture 04: 8086 Instruction Set and Assembly Environment

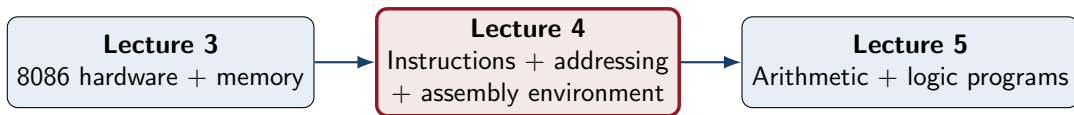
Afzal Hossan

Lecturer

Department of Computer Science and Engineering

Presidency University

Where Lecture 4 Fits



Today's bridge

Lecture 3 answered *what resources exist*. Today we learn *how instructions name those resources and how source code becomes executable code*.

Learning Outcomes

By the end of this laboratory lecture, students should be able to:

- ① classify the major groups of 8086 instructions;
- ② explain the conceptual fields of an 8086 machine instruction;
- ③ identify legal source/destination combinations for common data-transfer instructions;
- ④ recognize and apply the principal 8086 addressing modes;
- ⑤ distinguish instruction, directive, label, constant and comment;
- ⑥ describe edit → assemble → link → load → execute;
- ⑦ trace simple instructions and observe register/memory changes with a debugger;
- ⑧ write, assemble, execute and debug a small 8086 program.

Warm-up: What Does the CPU See?

Human-readable source

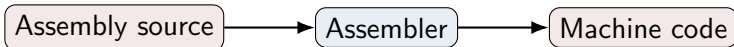
`MOV AX, BX`

“Copy the 16-bit value in BX into AX.”

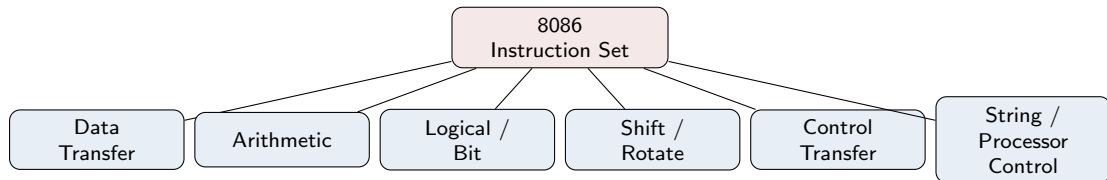
Processor-readable form

`8B C3`

Machine-code bytes (hexadecimal) interpreted by 8086 hardware.



The 8086 Instruction Set: Big Picture



Scope boundary

We identify all major families today. Detailed arithmetic/logical programming, flag-based decisions, shifts, loops, arrays, strings and procedures are developed in later laboratory classes.

Instruction Anatomy: Source-Level View

Label optional	Mnemonic operation	Operand(s) data / location	Comment explanation
-------------------	-----------------------	-------------------------------	------------------------

START: MOV AX, BX ; copy BX into AX

- Mnemonic tells *what* operation is required.
- Operands tell *where the data comes from and/or goes*.
- Labels give symbolic names to addresses; comments are ignored by the assembler.

Instruction Anatomy: Machine-Code View

8086 instructions are **variable length** (commonly 1–6 bytes). Depending on the instruction, encoding may contain:

Opcode + D/W bits	MOD-REG-R/M byte	Displacement 0, 8 or 16 bits	Immediate data 0, 8 or 16 bits
----------------------	---------------------	---------------------------------	-----------------------------------

- **Opcode**: operation to perform.
- **D**: direction in many register/memory encodings; **W**: byte/word size.
- **MOD**: addressing form; **REG**: register/opcode field; **R/M**: register or effective-address form.
- Displacement and immediate fields appear only when required.

Why Variable-Length Instructions?

Compact cases

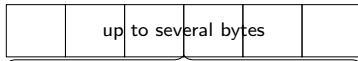
INC AX can be encoded very compactly because AX and the operation are implied by the opcode pattern.

Flexible cases

MOV [BX+SI+10H], 1234H must represent an operation, an effective address, a displacement and immediate data.

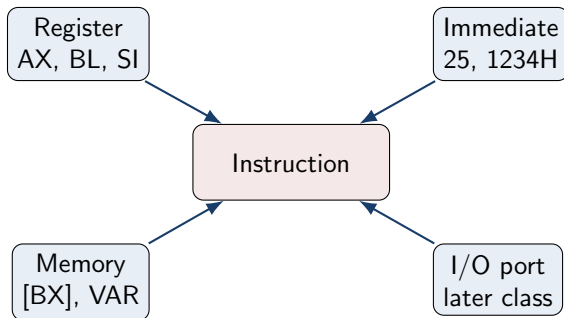
Key idea

Assembly syntax is simple; the assembler chooses the correct binary encoding. Students normally do not hand-encode every instruction, but understanding the fields explains addressing and instruction size.



encoding depends on operands

Operands: Where Can Data Come From?



Operand size matters: 8-bit data must normally interact with an 8-bit operand; 16-bit data with a 16-bit operand. The assembler catches many size mismatches before execution.

Data Transfer Instructions: Core Set

Instruction	Purpose	Example / note
MOV	Copy source to destination	MOV AX,BX; source unchanged
PUSH /	Transfer word to/from stack	stack details in Lecture 10
POP		
XCHG	Exchange two operands	XCHG AX,BX
LEA	Load effective offset address	LEA SI,ARRAY
LDS / LES	Load far pointer into register + DS/ES	advanced pointer transfer
IN / OUT	Transfer data through I/O port	interfacing focus later
LAHF /	FLAGS low byte $\leftrightarrow$ AH	flag transfer
SAHF		
PUSHF /	FLAGS $\leftrightarrow$ stack	word transfer
POPF		

Today's practical emphasis

MOV, simple register/memory transfers, and recognizing the role of the remaining transfer instructions.

MOV: The Most Important First Instruction



After execution: AX = 1234H, BX = 1234H. It is a **copy**, not a destructive move.

General form

MOV destination, source

MOV: Legal and Illegal Forms

Typical legal forms

- `MOV AX,BX`
- `MOV AL,25H`
- `MOV AX,1234H`
- `MOV AX,[BX]`
- `MOV [SI],AL`
- `MOV DS,AX`

Important restrictions

- No general **memory-to-memory** MOV.
- Cannot load a segment register directly from immediate data: `MOV DS,1234H` is invalid.
- CS cannot be destination of ordinary MOV.
- Operand sizes must agree.

Fixing memory-to-memory transfer

`MOV AX,[SRC]` then `MOV [DST],AX`. A register acts as the intermediate storage.

Arithmetic Instructions: Orientation

ADD / ADC

SUB / SBB

INC / DEC

MUL / IMUL

DIV / IDIV

- Addition/subtraction families update status flags and support multiword arithmetic via carry/borrow.
- Multiply/divide use implicit accumulator-related registers depending on operand size.
- CMP performs subtraction for flags but does not store the result.

Next lecture

Lecture 5 develops these operations in programs, including FLAGS analysis. Today: identify purpose and syntax only.

Logical Instructions: Orientation

Bitwise operations

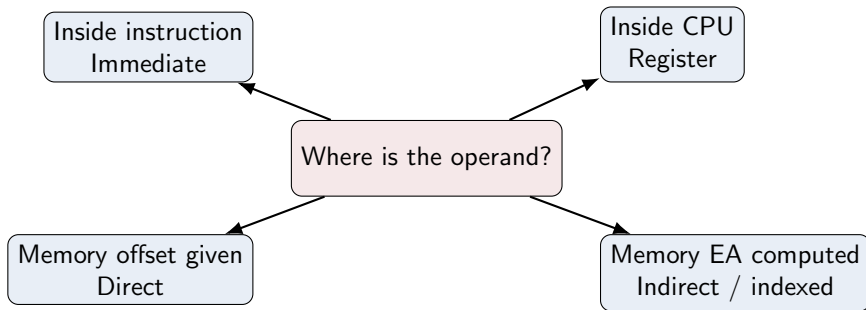
- AND: clear selected bits / masking
- OR: set selected bits
- XOR: toggle selected bits; XOR AX,AX clears AX
- NOT: invert every bit
- TEST: AND for flags; result not stored

1010	1100
0000	1111
0000	1100
AND mask	

Why learn them?

Logical operations are essential for control registers, device bits, masks and low-level embedded programming; detailed practice follows in Lectures 5–6.

Addressing Mode = How an Operand Is Located



Effective Address (EA) is the 16-bit offset computed for a memory operand. The segment base from Lecture 3 combines with EA to form the physical address.

Addressing Modes: Road Map

Mode	Example	Operand location / EA
Immediate	MOV AX, 1234H	data is in instruction
Register	MOV AX, BX	CPU register
Direct memory	MOV AL, [1234H]	$EA = 1234H$
Register indirect	MOV AX, [BX]	$EA = BX$
Based	MOV AX, [BP+08H]	$EA = BP + 08H$
Indexed	MOV AX, [SI+10H]	$EA = SI + 10H$
Based-indexed	MOV AX, [BX+DI]	$EA = BX + DI$
Based-indexed disp.	+ MOV AX, [BP+SI+20H]	$EA = BP + SI + 20H$

Immediate Addressing



- Operand value is part of the instruction itself.
- No extra memory read is needed to obtain the source operand.
- Constants are convenient, but the value is fixed until code is changed.

Examples: `MOV BL, 7FH`, `MOV CX, 1000`, `ADD AX, 5` (arithmetic detail later).

Register Addressing



- Both operands are registers; no data-memory access for the operand.
- Usually compact and fast.
- 8-bit registers: `AL`, `AH`, `BL`, `BH`, `CL`, `CH`, `DL`, `DH`.
- 16-bit general/index/pointer registers can participate where the instruction allows.

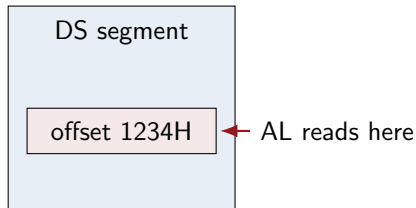
Direct Memory Addressing

Example

```
MOV AL, [1234H]
```

$EA = 1234H$. By default this data reference uses DS, so physical address is

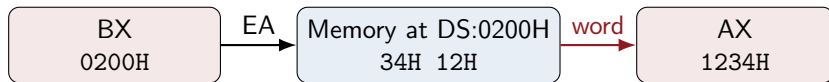
$DS \times 10H + 1234H$.



Brackets mean memory

`MOV AX, 1234H` loads the number 1234H. `MOV AX, [1234H]` loads a word stored at memory offset 1234H.

Register-Indirect Addressing



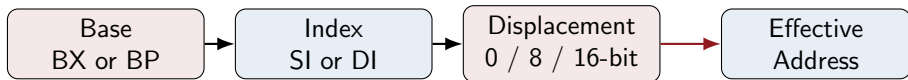
Example: `MOV AX, [BX]`. Valid address-holding registers for 8086 memory addressing are built from **BX/BP** and/or **SI/DI** in legal combinations; **AX, CX, DX** and **SP** are not general R/M effective-address registers in 8086 encoding.

Base and Index Registers: Roles



- **BX**: commonly points into data in DS.
- **BP**: commonly addresses stack-frame data; default segment is SS when BP participates in EA.
- **SI/DI**: naturally support indexed data and later string/array operations.
- A displacement can be added to these forms to reach a field/element.

8086 Effective-Address Building Blocks



$$EA = \text{Base} + \text{Index} + \text{Displacement}$$

Not every term must be present. Legal 8086 base+index pairs are BX+SI, BX+DI, BP+SI, BP+DI. Forms such as BX+BP or SI+DI are not encodable as a single 8086 EA.

Based Addressing

Example 1

MOV AX, [BX+10H]

$EA = BX + 10H$

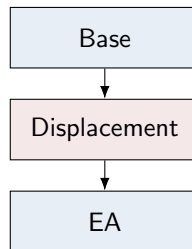
Default segment: DS

Example 2

MOV AX, [BP+10H]

$EA = BP + 10H$

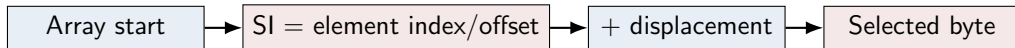
Default segment: SS



Indexed Addressing

MOV AL, [SI+05H]

$$EA = SI + 05H$$



This pattern becomes especially useful for arrays and tables. Detailed array processing is intentionally reserved for Lecture 9.

Based-Indexed Addressing

`MOV AX, [BX+DI]` $EA = BX + DI$

BX = base of structure

→ DI = index within data

→ $EA = BX + DI$

With displacement: `MOV AX, [BX+DI+20H]` gives $EA = BX + DI + 20H$.

Concept

Base can locate a data structure; index can select an element; displacement can select a field or add a fixed offset.

Default Segment Selection

Effective-address form	Default segment	Typical meaning
BX, SI, DI based forms	DS	ordinary data
BX+SI / BX+DI	DS	data/array structures
BP based forms	SS	stack-frame parameters/local data
BP+SI / BP+DI	SS	indexed stack-related data

Segment override

A segment-override prefix can explicitly choose another segment when needed, e.g. `MOV AX, ES:[DI]`. Use overrides deliberately; the default rules make normal code shorter.

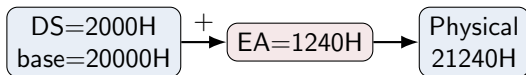
Address Calculation Example

Suppose DS=2000H, BX=1200H, SI=0030H; instruction: MOV AX, [BX+SI+10H].

$$EA = 1200H + 0030H + 0010H = 1240H$$

$$\text{Physical Address} = DS \times 10H + EA$$

$$= 20000H + 1240H = 21240H$$



Quick Challenge: Name the Addressing Mode

- 1 MOV AX, 25H
- 2 MOV DX, AX
- 3 MOV AL, [1234H]
- 4 MOV AX, [BX]
- 5 MOV AX, [BP+08H]
- 6 MOV AL, [SI+04H]
- 7 MOV AX, [BX+DI]
- 8 MOV AX, [BP+SI+20H]

Quick Challenge: Name the Addressing Mode

- 1 MOV AX,25H
- 2 MOV DX,AX
- 3 MOV AL,[1234H]
- 4 MOV AX,[BX]
- 5 MOV AX,[BP+08H]
- 6 MOV AL,[SI+04H]
- 7 MOV AX,[BX+DI]
- 8 MOV AX,[BP+SI+20H]

Answers

Immediate; Register; Direct memory; Register indirect; Based + displacement; Indexed + displacement; Based-indexed; Based-indexed + displacement.

Common Addressing Mistakes

Wrong / misleading

- `MOV AX, [CX]`
- `MOV AX, [BX+BP]`
- `MOV AX, [SI+DI]`
- `MOV [1000H], [2000H]`
- forgetting that BP defaults to SS

Better thinking

- use BX/BP/SI/DI legal EA forms
- use a register between two memory locations
- calculate EA first
- then apply default segment
- then calculate physical address if asked

Assembly Language: What Is in a Source File?

Instructions
CPU executes

Directives
assembler obeys

Labels
symbolic addresses

Comments
humans read

Critical distinction

Instructions become machine operations. **Directives** guide the assembler and usually do not represent CPU instructions themselves.

General Assembly Statement Format

`[label:] mnemonic [operands] [; comment]`

Part	Example	Meaning	
Label	START:	symbolic address for code/data	
Mnemonic	MOV	operation name	
Operands	AX,BX	destination/source or instruction parameters	Whitespace
Comment	; copy BX	ignored by assembler; documents intent	

separates tokens; commas separate operands; syntax details can vary slightly among MASM/TASM/EMU8086-style assemblers.

Identifiers, Labels and Constants

- Choose meaningful names: COUNT, TOTAL, LOOP_START.
- A **label** represents an address determined by the assembler.
- A symbolic constant represents a fixed value and improves readability.
- Avoid reserved instruction/register names as user identifiers.
- Consistent naming and comments make debugging easier than cryptic one-letter names.

Why labels matter

JMP AGAIN is maintainable because the assembler recalculates the target if code moves; hard-coded numeric branch addresses are fragile. Branching itself is covered later.

Assembler Directives: Categories

Category	Typical directives	Purpose
Data definition	DB, DW	reserve/initialize byte or word data
Segments/model	.MODEL, .DATA, .CODE, SEGMENT/ENDS	organize program sections (assembler-dependent)
Procedure/end	PROC, ENDP, END	mark program/procedure structure
Symbol definition	EQU	define symbolic constant
Address assistance	ASSUME	tell assembler expected segment-register relationships

Remember

Directives are commands to the **assembler**, not operations executed by the 8086 ALU/control unit. Exact directive syntax depends on the assembler used in the lab.

DB and DW: Defining Data

Byte data

```
NUM DB 25H  
ARR DB 10H,20H,30H  
MSG DB 'HELLO'
```

Word data

```
VALUE DW 1234H  
LIST DW 1000H,2000H
```

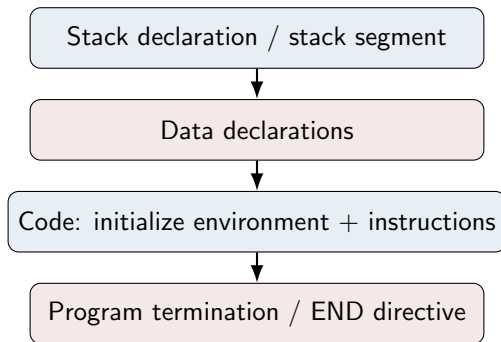
8086 stores word bytes in little-endian order.

DW 1234H



low address high address

Program Organization: Conceptual View



The exact wrapper differs between COM-style, EXE-style, MASM/TASM and emulator templates. Focus on the roles: **data**, **code**, correct segment setup, and a valid termination path.

A Small MASM/TASM-Style 8086 Program

```
.MODEL SMALL
.STACK 100H
.DATA
NUM1 DW 1234H
COPY DW ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV AX, NUM1
    MOV COPY, AX
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

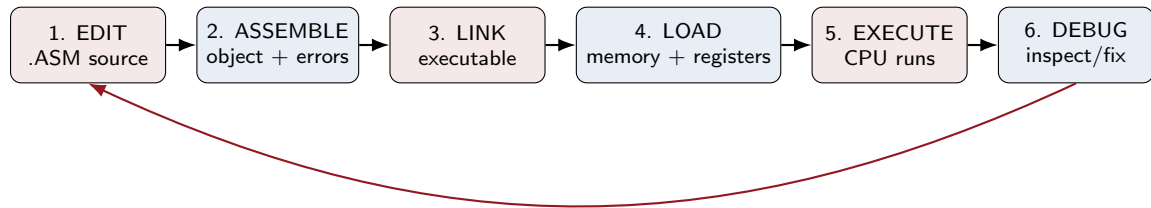
Read the Program Like a Machine

- ➊ `MOV AX,@DATA`: obtain data-segment value through assembler/linker support.
- ➋ `MOV DS,AX`: initialize DS; segment register cannot be loaded directly from immediate data.
- ➌ `MOV AX,NUM1`: read word from data memory into AX.
- ➍ `MOV COPY,AX`: write AX to memory.
- ➎ `MOV AH,4CH` and `INT 21H`: DOS-style program termination in a classic teaching environment.

After the two copy instructions

AX = 1234H and variable COPY = 1234H. This is enough to practice data movement without stealing Lecture 5's arithmetic work.

From Source Code to Running Program

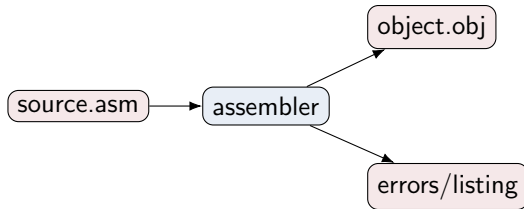


Development is a loop

Errors are normal evidence. Syntax errors return to editing; wrong results require tracing and reasoning; a corrected program is rebuilt and tested again.

What the Assembler Actually Does

- Parses mnemonics, operands, labels and directives.
- Checks many syntax and operand-size/addressing constraints.
- Builds a **symbol table** for labels and variables.
- Converts instructions to machine code.
- Resolves what it can and records relocatable/external references for the linker.
- Produces diagnostics and, depending on tool, listing/object files.



Assembler vs Linker vs Loader

Tool	Input → output	Main responsibility
Assembler	source → object	translate assembly; symbols; syntax checking
Linker	object(s)/libraries → executable	combine modules, resolve external references, arrange executable image
Loader	executable → memory	place program in memory, establish execution environment, start program
Debugger	running program	step, inspect, modify/test state, find logic errors

Errors: Three Different Stages

Assembly-time

MOV AX,BL
size mismatch
illegal operand/addressing
misspelled mnemonic

Link-time

undefined external symbol
missing module/library
duplicate symbols

Run-time / logic

program executes but wrong
value
wrong address
unexpected control flow

Debugging mindset

First identify *which stage* failed. That determines whether to inspect syntax, linking configuration, or execution state.

Debugger: What Should Students Observe?

Registers
AX BX CX DX...

Segments + IP
CS DS SS ES IP

FLAGS
status/control

Memory
bytes/words

- Single-step one instruction at a time.
- Record state **before** and **after**.
- Verify the destination changed as predicted and source remained/changed as defined.
- For memory operands, compute EA and confirm the inspected memory location.

Trace Example: Register and Memory Changes

Assume initially AX=0000H, BX=1234H, DS=2000H, and word at DS:0100H = ABCDH.

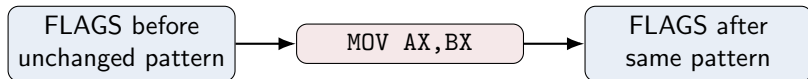
Instruction	AX after	Explanation
MOV AX,BX	1234H	register source copied
MOV AX,5678H	5678H	immediate source copied
MOV AX,[0100H]	ABCDH	word read from memory

Lab habit

Predict first, step second, compare third. A debugger is a verification tool, not a substitute for understanding.

Flags and MOV: An Important Observation

Ordinary MOV does **not** modify the arithmetic status flags. This makes it ideal for learning the debugger: you can focus on data location and movement without flag changes.



Arithmetic/logical instructions often affect flags; that becomes central in Lecture 5.

Laboratory Experiment: Objectives

Experiment: Writing, assembling and executing basic 8086 assembly programs.

- ➊ Open the available 8086 assembler/emulator/debugger environment.
- ➋ Enter a minimal valid source program.
- ➌ Assemble/build and correct syntax errors.
- ➍ Execute register-to-register and immediate-to-register MOV.
- ➎ Define byte/word data and transfer between memory and registers.
- ➏ Single-step instructions and record register/memory changes.
- ➐ Identify the addressing mode of every tested instruction.
- ➑ Deliberately create one invalid instruction, read the diagnostic, and correct it.

Lab Task A: Register Transfers

Program fragment

```
MOV AX,1234H  
MOV BX,AX  
MOV CX,BX
```

Before running, predict: AX, BX, CX after each line.



Record: instruction, addressing mode, before-state, after-state. Confirm that copying does not erase the source.

Lab Task B: Memory Transfers

Define: NUM DW 55AAH and COPY DW ?. Then execute:

```
MOV AX,NUM    MOV COPY,AX
```

- Inspect memory bytes used by NUM: expect AAH at the lower address and 55H at the next address.
- Step MOV AX,NUM; verify AX becomes 55AAH.
- Step MOV COPY,AX; verify the destination word in memory.
- Explain why a direct memory-to-memory MOV COPY,NUM is not the normal 8086 MOV form.

Lab Task C: Addressing-Mode Practice

Test or analyze these forms using safe initialized addresses/data:

- ➊ `MOV AX,1234H` — immediate
- ➋ `MOV DX,AX` — register
- ➌ `MOV AL,[offset]` — direct memory
- ➍ `MOV AX,[BX]` — register indirect
- ➎ `MOV AX,[BX+10H]` — based + displacement
- ➏ `MOV AL,[SI+02H]` — indexed + displacement
- ➐ `MOV AX,[BX+DI]` — based-indexed

For each memory form, write the EA formula and identify the default segment.

Lab Task D: Learn from an Error

Try to assemble each line separately and explain the diagnostic:

Intentional errors

```
MOV AX,BL
MOV DS,1234H
MOV AX,[CX]
MOV [1000H],[2000H]
```

Repair strategy

- Match operand sizes.
- Use AX as intermediate for DS.
- Use a legal EA register.
- Use a register between memory operands.

Learning goal

Do not memorize only “valid/invalid.” State the architectural or encoding reason behind each restriction.

A Practical Debugging Checklist

- 1 Did the source assemble without errors/warnings?
- 2 Is DS initialized before accessing declared data in an EXE-style program?
- 3 Are operand sizes compatible?
- 4 Are brackets used only when memory contents are intended?
- 5 Is the effective-address combination legal on 8086?
- 6 Is BP causing SS to be selected when DS was intended?
- 7 Before stepping, what exact register/memory change do you predict?
- 8 After stepping, does the observed state match? If not, locate the first instruction where prediction and reality diverge.

Mini Case: Decode the Programmer's Intent

A student wants to copy the word at DS:0200H to DS:0300H. Which sequence is valid?

Option A

```
MOV [0300H], [0200H]
```

Option B

```
MOV AX, [0200H]  
MOV [0300H], AX
```

Mini Case: Decode the Programmer's Intent

A student wants to copy the word at DS:0200H to DS:0300H. Which sequence is valid?

Option A

```
MOV [0300H], [0200H]
```

Option B

```
MOV AX, [0200H]  
MOV [0300H], AX
```

Reasoning

Option B. General 8086 MOV does not allow both operands to be ordinary memory locations. AX temporarily carries the word.

Mini Case: Brackets Change Meaning

Compare:

MOV AX, 1234H vs. MOV AX, [1234H]

Without brackets

AX receives the **constant** 1234H.

Addressing: immediate.

With brackets

AX receives the **word stored in memory** at offset 1234H (default DS).

Addressing: direct memory.

One pair of brackets changes the source from a number to a memory location.

Mini Case: Which Segment Is Used?

Assume DS=1000H, SS=3000H, BP=0100H. For MOV AX, [BP+20H]:

$$EA = 0100H + 0020H = 0120H$$

Because BP participates, default segment is **SS**:

$$Physical = 3000H \times 10H + 0120H = 30120H$$

Common mistake

Using DS automatically for every memory operand. Segment selection depends on the EA form; BP-based addressing defaults to SS.

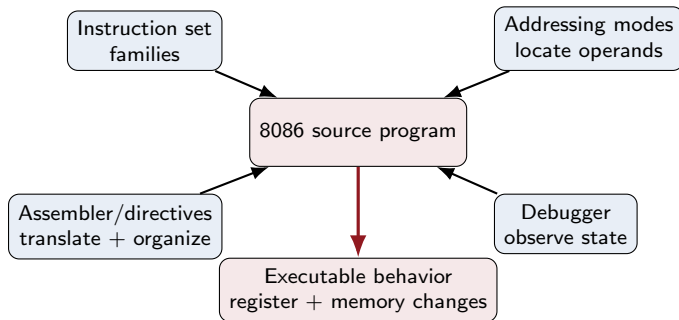
Knowledge Check

- ❶ What is the difference between an instruction and a directive?
- ❷ Why can `MOV AX, [BX]` work but `MOV AX, [CX]` cannot on 8086?
- ❸ Which default segment is selected by a BP-based EA?
- ❹ What does the assembler do that the linker does not?
- ❺ What is the difference between `MOV AX, 1000H` and `MOV AX, [1000H]`?
- ❻ Why is single-step execution useful?
- ❼ Which Lecture 4 topics are deliberately not developed into full arithmetic/logic programs today?

Knowledge Check: Answers

- ➊ Instruction becomes an operation for the CPU; directive controls assembly/program organization.
- ➋ 8086 R/M encoding supports BX/BP/SI/DI-based EA forms, not CX as an address register.
- ➌ SS.
- ➍ Assembler translates source/object symbols; linker combines objects and resolves external/relocatable relationships into an executable.
- ➎ First loads immediate 1000H; second reads a word from memory offset 1000H.
- ➏ It exposes state changes instruction by instruction, locating the first wrong step.
- ➐ Detailed arithmetic/logical programming and FLAGS-based computation are Lecture 5; shifts/bit manipulation, branching, arrays, stack/strings follow later.

Lecture 4 Concept Map



Takeaways

- 8086 instructions combine an operation with operands located through specific addressing modes.
- Variable-length machine encoding provides compact simple operations and flexible memory addressing.
- MOV is the foundation for learning legal operand combinations and data flow.
- Effective address is built from legal base/index/displacement combinations; BP-based forms default to SS, most others to DS.
- Instructions and assembler directives are fundamentally different.
- A program moves through edit, assemble, link, load, execute and debug stages.
- The best lab habit is: **predict** → **single-step** → **observe** → **explain**.

Lecture 05

Arithmetic and Logical Programming in 8086

ADD / ADC

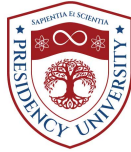
SUB / SBB

MUL / DIV

AND / OR / XOR

FLAGS

Bring today's addressing-mode and debugger skills: they become the foundation for real computational programs.



Thank You
Questions?

Afzal Hossan

Lecturer, Department of Computer Science and Engineering
Presidency University